

```

use std::env::args;
use std::process::exit;
use std::str::FromStr;
use std::fmt;
use std::ops;
use std::cmp::Ordering;

fn usage() -> ! {
    println!("Usage: ./fraction-math operand1 operator operand2");
    exit(0);
}

fn parse_int<T: FromStr>(s: &str) -> Result<T, <T as FromStr>::Err> {
    s.trim().parse::<T>()
}

fn parse_fraction(s: &str) -> Option<Fraction> {
    // Parse numerator, denominator, and remainder
    let parts: Vec<Result<i32, <i32 as FromStr>::Err>> = s.split('/').
        .map(parse_int)
        .collect();

    // Return None if numerator or denominator invalid or there was a
    remainder
    // Return numerator with a denominator of 1 if just numerator
    // Otherwise numerator and denominator
    let num_parts = parts.len();
    match parts.iter().any(|s| s.is_err()) || num_parts > 2 {
        true => None,
        false => {
            let num: i32 = parts[0].clone().unwrap();
            let den: i32 = if num_parts < 2 { 1 } else {
                parts[1].clone().unwrap();
            };
            Some(Fraction::new(num, den))
        },
    }
}

#[derive(Copy, Clone)]
struct Fraction {
    num: i32,
    den: i32,
}

impl Fraction {
    // Create new fraction
    fn new(num: i32, den: i32) -> Self {
        Self {num: num, den: den}
    }

    // Fraction comparison:
    // n1/d1 OP n2/d2 = n1*d2 OP n2*d1
    fn compare(&self, other: &Self) -> i32 {
        self.num * other.den - other.num * self.den
    }
}

```

```

    }
}

impl fmt::Debug for Fraction {
    // Show fraction
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}/{}", self.num, self.den)
    }
}

impl ops::Add for Fraction {
    type Output = Self;

    // Add fractions:
    //  $n_1/d_1 + n_2/d_2 = (n_1*d_2 + n_2*d_1) / (d_1*d_2)$ 
    fn add(self, other: Self) -> Self {
        reduce(
            self.num * other.den + other.num * self.den,
            self.den * other.den
        )
    }
}

impl ops::Sub for Fraction {
    type Output = Self;

    // Subtract fractions:
    //  $n_1/d_1 - n_2/d_2 = (n_1*d_2 - n_2*d_1) / (d_1*d_2)$ 
    fn sub(self, other: Self) -> Self {
        reduce(
            self.num * other.den - other.num * self.den,
            self.den * other.den
        )
    }
}

impl ops::Mul for Fraction {
    type Output = Self;

    // Multiply fractions:
    //  $n_1/d_1 * n_2/d_2 = (n_1*n_2) / (d_1*d_2)$ 
    fn mul(self, other: Self) -> Self {
        reduce(self.num * other.num, self.den * other.den)
    }
}

impl ops::Div for Fraction {
    type Output = Self;

    // Divide fractions:
    //  $(n_1/d_1) / (n_2/d_2) = (n_1*d_2) / (n_2*d_1)$ 
    fn div(self, other: Self) -> Self {
        reduce(self.num * other.den, self.den * other.num)
    }
}

```

```

}

impl PartialOrd for Fraction {
    // Fraction comparison for ordering
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.compare(other).cmp(&0))
    }
}

impl PartialEq for Fraction {
    // Fraction comparison for equality
    fn eq(&self, other: &Self) -> bool {
        self.compare(other) == 0
    }
}

// Fraction reduction
fn reduce(num: i32, den: i32) -> Fraction {
    if den == 0 {
        panic!("Division by 0");
    }

    // Reduce by dividing numerator and denominator by greatest common
    denominator,
    // and adjust sign of numerator and denominator as follows:
    //
    // n  d  sign n  sign d
    // +  +      +      +
    // +  -      -      +
    // -  +      -      +
    // -  -      +      +
    let den_sign: i32 = if den > 0 { 1 } else { -1 };
    let g: i32 = gcd(num, den);
    Fraction::new(den_sign * num / g, den.abs() / g)
}

// Greatest common denominator
fn gcd(a: i32, b: i32) -> i32 {
    let mut a: i32 = a.abs();
    let mut b: i32 = b.abs();
    while b != 0 {
        (a, b) = (b, a % b);
    }

    a
}

// Fraction result
enum FractionResult {
    Frac(Fraction),
    Bool(bool),
}

impl fmt::Debug for FractionResult {

```

```

// Show fraction or boolean
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self {
        Self::Frac(fraction) => write!(f, "{fraction:?}"),
        Self::Bool(boolean) => write!(f, "{}", *boolean as u8),
    }
}

// Do fraction math
fn fraction_math(f1: Fraction, f2: Fraction, op: String) ->
FractionResult {
    let op: &str = &op[..];
    match op {
        "+" => FractionResult::Frac(f1 + f2),
        "-" => FractionResult::Frac(f1 - f2),
        "*" => FractionResult::Frac(f1 * f2),
        "/" => FractionResult::Frac(f1 / f2),
        ">" => FractionResult::Bool(f1 > f2),
        ">=" => FractionResult::Bool(f1 >= f2),
        "<" => FractionResult::Bool(f1 < f2),
        "<=" => FractionResult::Bool(f1 <= f2),
        "==" => FractionResult::Bool(f1 == f2),
        "!=" => FractionResult::Bool(f1 != f2),
        _ => panic!("Invalid operation {op}")
    }
}

fn main() {
    let mut args = args().skip(1);

    // Parse 1st command-line argument
    let frac1 = args
        .next()
        .and_then(|s| parse_fraction(&s))
        .unwrap_or_else(|| usage());

    // Get 2nd command-line argument
    let op = args
        .next()
        .unwrap_or_else(|| usage());

    // Parse 3rd command-line argument
    let frac2 = args
        .next()
        .and_then(|s| parse_fraction(&s))
        .unwrap_or_else(|| usage());

    // Do fraction math and show result
    let result: FractionResult = fraction_math(frac1, frac2, op);
    println!("{result:?}");
}

```